PRISM: Compositional Reward Modeling for Long-Horizon Manipulation with Structured Progress Supervision

Zi-ang Cao[1,2,*], Kaiyuan Zheng[1,3,*], Yinzhen Xu[1], Haonan Duan[1], Jiasheng Gu[1], Fengyuan Hu[1], Nan Zhu[1], Kun Lei[1], Kevin Lin[1], Ruijie Zheng[1], Shenyuan Gao[1], Fangqi Zhu[1], Zhe Zhang[1], Radha Poovendran[3], Linxi Fan[1,†], Guanzhi Wang[1,†], Yuke Zhu[1,2,†]

[1]NVIDIA [2]The University of Texas at Austin [3]University of Washington

[*]Equal contribution · [†]Project leads

https://robot-reward-prism.github.io/

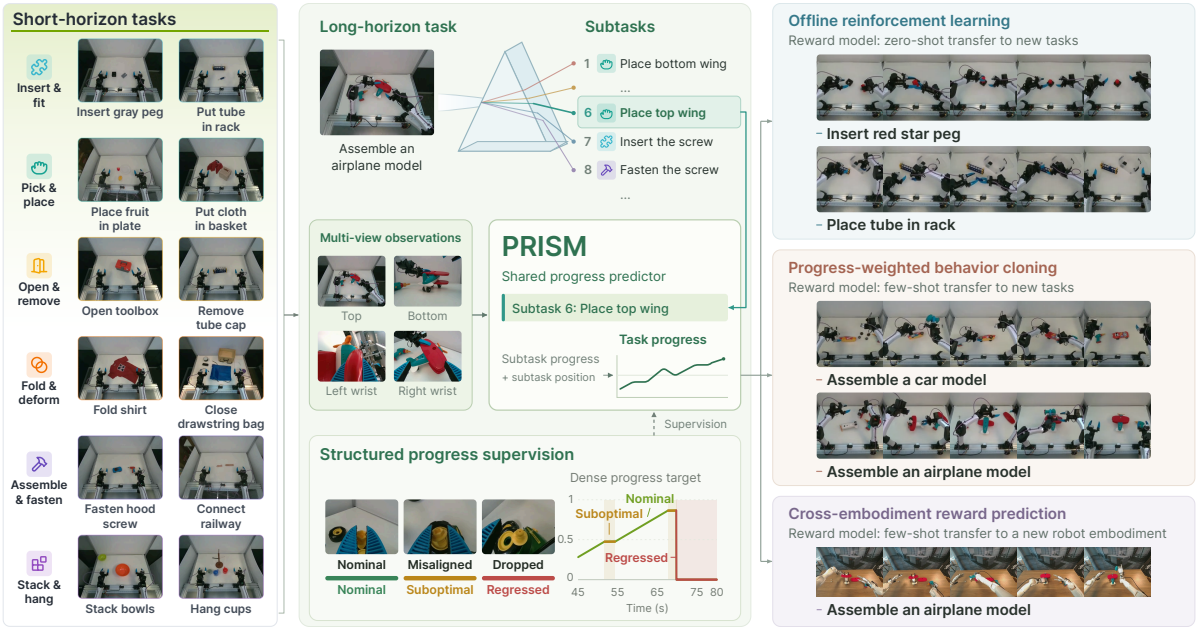


Figure 1: **Overview of PRISM.** Left: diverse short-horizon tasks for pretraining. Center: subtask descriptions condition a progress predictor, and sparse annotations define dense progress supervision. Right: offline RL, progress-weighted cloning after long-task adaptation, and cross-embodiment adaptation.

Abstract

Robot reward modeling faces two challenges: generalizing across tasks and accurately tracking progress in long-horizon manipulation. General-purpose reward models demonstrate broad task transfer, but transfer from short-task pretraining to long-horizon tasks remains underexplored. Long-horizon progress models often rely on task-specific subtask classifiers or fixed action vocabularies, limiting the reuse of learned progress knowledge when adapting to new tasks. To address these challenges, we introduce PRISM, a vision-language robot reward model that learns to identify unfinished subtasks and estimate their continuous progress using open-vocabulary subtask descriptions. We train a language-conditioned model to estimate progress in short-horizon tasks and individual subtasks of long-horizon tasks. For long-horizon tasks, we combine the selected subtask's predicted progress with credit for preceding completed subtasks to estimate whole-task progress. General progress prediction requires training the model across diverse tasks with progress supervision that accounts for imperfect execution. Elapsed-time proxies can misrepresent progress when relative subtask durations vary, or failures reverse local progress. We therefore introduce an annotation protocol that converts sparse subtask boundaries and action-quality annotations into dense progress targets, without requiring annotators to assign numerical progress scores. Pretraining with this supervision on 200 diverse, mostly short-horizon manipulation tasks enables zero-shot reward prediction on unseen tasks and improves progress prediction after adaptation to multi-minute, long-horizon tasks. Experiments demonstrate (i) generalizable reward prediction, (ii) benefits from pretraining and composing subtask progress estimates, and (iii) improved downstream policy performance.

© 2026 NVIDIA. All rights reserved.

1. Introduction

Progress-based reward models estimate how much of a task has been completed, providing learning signals for robot policies [1, 2]. For long-horizon manipulation, these models should reuse progress knowledge learned from shorter tasks while tracking failed attempts and recovery.

Consider assembling a model airplane by placing a wing, inserting a screw, and tightening it with a drill. Completing wing placement should receive the same progress credit across executions, even when the time spent on each subtask varies. During insertion, repeated misalignment should slow progress, while dropping the screw should reset insertion progress without erasing credit for the placed wing. Furthermore, reaching for the drill should not advance progress while insertion remains unfinished. These distinctions motivate us to compose whole-task progress from subtask progress, and to combine subtask boundaries with action-quality annotations to capture setbacks and recovery.

General-purpose reward models learn from diverse robot videos [2--4] or use pretrained vision-language models directly [5] to provide reward signals across diverse tasks. We ask whether learning progress at the subtask level improves whole-task progress tracking and supports transfer from short tasks to long-horizon tasks. Some long-horizon progress models use task-specific subtask classifiers [1] or a fixed action vocabulary shared across tasks [6]. These choices limit reuse of learned progress knowledge when subtasks change in a new long-horizon task. We seek a model that learns subtask selection and progress prediction through language descriptions, so that knowledge from short tasks can be reused in longer compositions.

Progress supervision poses a second challenge. Subtask completion keyframes alone do not reflect failed attempts or recovery. Local execution judgments can describe setbacks [6, 7], but a progress curve must also show how much work is lost. A brief failure may undo the current attempt without undoing earlier subtasks.

We introduce PRISM, a vision-language robot reward model that learns to select the unfinished subtask and predict its local progress from open-vocabulary descriptions (Fig. 1). We combine this prediction with credit for preceding completed subtasks to estimate whole-task progress. Incorrect subtask choices can distort this whole-task estimate, so training considers subtask order to define a local-progress target for each selected subtask. The same language interface lets us reuse the pretrained model on new tasks and longer compositions without replacing model components.

To train this predictor, we combine subtask boundaries with action-quality annotations to construct a dense progress curve without direct numerical scoring. Subtask boundaries mark when subtasks are completed, while action-quality annotations describe nominal, suboptimal, and regressed execution within each subtask. The constructed progress curve advances more slowly during suboptimal execution and drops when regressed execution loses local progress. It rises again during recovery while preserving credit for earlier completed subtasks. Using this supervision, we pretrain PRISM on 200 mostly short manipulation tasks.

We evaluate how these design choices affect progress prediction on new tasks and a different embodiment, as well as downstream policy learning. We also show that structured supervision improves progress predictions from a prior vision-language model (VLM)- based reward model. We further show PRISM ’s generalization capabilities, achieving 20.7% lower progress RMSE than the strongest evaluated baseline without target-task reward labels or fine-tuning on 50 held-out tasks. On two 3--5-minute long-horizon tasks, PRISM fine-tuned with 15 labeled trajectories per task outperforms baselines adapted with 100; matched comparisons show that pretraining improves prediction on both tasks at both adaptation budgets. The reward model’s zero-shot progress prediction also improves offline reinforcement learning (RL) on held-out tasks. During iterative policy learning on the long tasks, progress-weighted behavior cloning improves success over unweighted behavior

cloning with the same parent policy and data.

Our contributions are threefold:

- **Compositional progress prediction.** We train the model to select subtasks and predict local progress, then compose their outputs into whole-task progress. During adaptation, task order defines progress targets for the model's own subtask choices.
- **Structured progress supervision.** We combine subtask boundaries and action-quality annotations to construct dense progress curves, with explicit rules for failure and recovery.
- **Evidence for reward model generalization and policy improvement.** We show that short-task pretraining improves long-task adaptation and that predicted progress supports downstream policy learning.

2. Related Work

General-purpose robot reward models

Pretrained visual and vision-language representations can be adapted to provide reusable reward signals from image goals, text, or demonstration videos [8--11]. Recent approaches use pretrained VLMs to estimate task progress without further fine-tuning [5, 12] or learn reward and value predictors from diverse video data [2--4, 13]. However, the benefits of short-task pretraining for long-horizon progress prediction remain underexplored, so we study whether subtask supervision improves long-task adaptation.

Progress supervision

Video-derived supervision for reward learning includes temporal ordering and transformations [3, 14, 15], signed frame distances or remaining-time targets [13, 16], and successful-transition prediction [17]. Other supervision uses subtask boundaries, interval-level progress and completion labels, or simulated failures [7, 18, 19]. To capture local setbacks and recovery while preserving completed work, we deterministically convert sparse subtask boundaries and action-quality judgments into dense progress curves without direct numerical scoring. The constructed curves reset local progress on failure and rebuild it according to action quality during retries, while retaining credit for completed subtasks.

Progress estimation through task decomposition

REDS infers subtasks through video--language alignment and learns subtask-conditioned rewards from segmented demonstrations [20]. SARM reports that REDS's predictions remain largely step-shaped despite monotonicity regularization, limiting fine-grained progress estimation. By separately predicting subtask identity and continuous local progress, SARM achieves lower progress-estimation error on T-shirt folding [1]. SARM2 extends subtask-aware reward modeling to multiple tasks [6]. PRM4RL composes subtask indices and local progress through generated simulator-state functions, while RoboMeter advances through VLM-proposed subtasks using success predictions during online RL [2, 21]. PRISM studies how short-task pretraining supports longer compositions by combining predicted local progress with credit for preceding completed subtasks. Its supervision explicitly captures local failure resets and recovery, while task order defines progress targets even for incorrect subtasks.

Reward-guided policy learning

Learned visual rewards already support robot policy improvement through RL [2, 14, 22]. Offline RL and advantage-weighted learning provide complementary policy-improvement tools [23--25]. We evaluate zero-shot reward models for offline RL on short tasks and adapted progress predictors for progress-weighted behavior learning on long-horizon tasks.

3. Method

PRISM learns to select the unfinished subtask and predict its local progress, then combines these outputs into whole-task progress. The same language-conditioned model is pretrained on mostly short tasks and reused for longer compositions. Its progress targets account for both execution quality and the evaluated subtask's position in the task sequence.

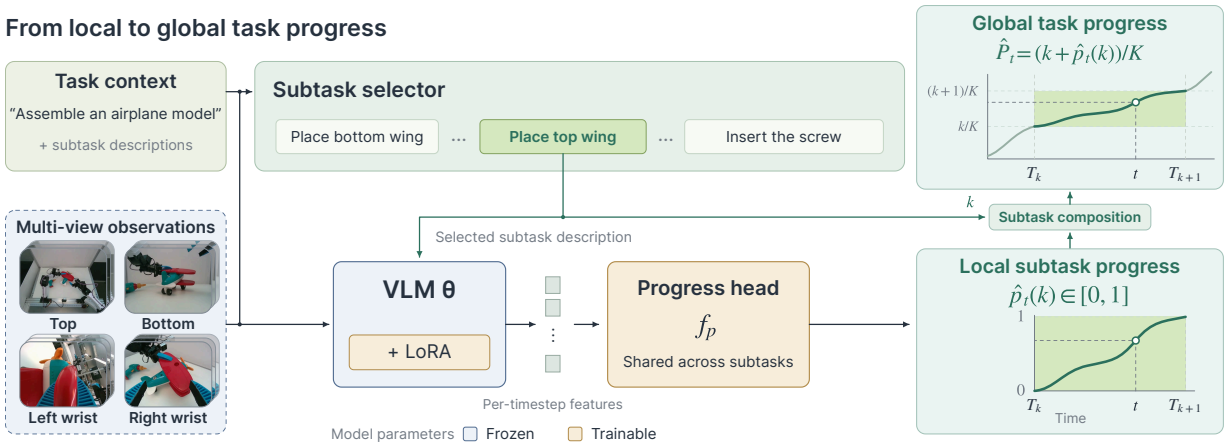


Figure 2: **From local to global task progress in PRISM.** PRISM learns subtask selection and local-progress prediction with a shared vision-language model. It selects the unfinished subtask from language descriptions, estimates progress for the selected description, and combines this estimate with credit for completed subtasks.

3.1. Predicting Progress through Subtasks

Given camera observations, a task description, and the task's subtask sequence, PRISM selects the unfinished subtask and predicts its local progress (Fig. 2). Our model learns both subtask selection and progress prediction, with a progress predictor conditioned on the selected subtask description. The selected subtask's position in the sequence and its local progress determine whole-task progress. Changing a task's subtask sequence does not require changing the model.

Let k_t be the selected subtask's zero-based index, and let $\hat{p}_t(k_t) \in [0, 1]$ be its predicted local subtask progress. For a task with K subtasks, we assign each subtask an equal share of global whole-task progress:

$$\hat{P}_t = \frac{k_t + \hat{p}_t(k_t)}{K}. \tag{1}$$

Six completed subtasks and halfway progress through the seventh give $6.5/9$ in a nine-subtask task. If local progress falls to zero, the estimate becomes $6/9$. This composition assumes that preceding subtasks remain completed after a local failure; an incorrect selection can still distort the estimate.

Shared predictor

In our model, subtask selection and progress prediction share a VLM that processes multiview observations sampled over time, providing context to distinguish a failed attempt from its recovery. A temporal transformer combines features for local-progress prediction through a shared head. Temporal aggregation can use later observations. In our policy experiments, we score recorded trajectories offline.

3.2. Supervision for Imperfect Execution

Selecting the right subtask does not reveal how much of it is complete. Elapsed time advances through both a successful insertion and repeated misalignment, yet these executions should receive different progress targets. We therefore combine subtask annotations with judgments of local action quality (AQ). Subtask annotations identify completed work. AQ judgments describe setbacks and recovery within the unfinished subtask.

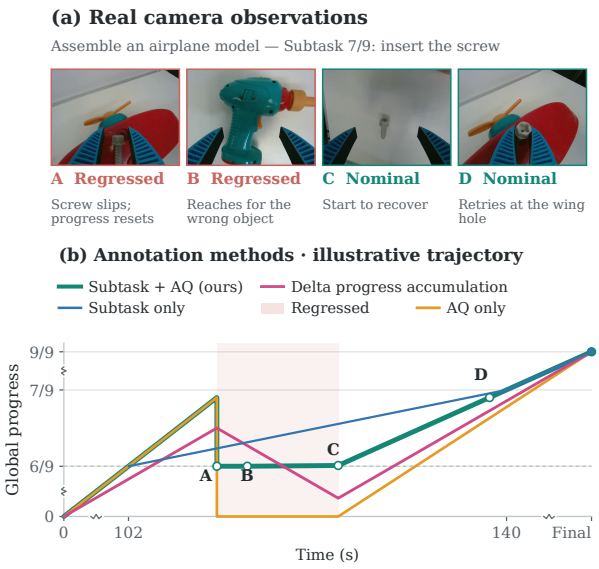


Figure 3: **Why combine subtask completion and action quality (AQ)?** Six of nine subtasks are complete before the screw-insertion failure at A--B and recovery at C--D (top). The illustrative curves (bottom) compare progress reconstruction from sparse annotations. Subtask only misses the failure, while AQ only drops to zero and discards completed-subtask progress. Delta progress accumulates positive, zero, or negative changes, following prior work [7]. The plotted curve falls gradually after the screw slips, leaving progress near its pre-failure value; without subtask completion anchors, prolonged regression could also drive it below $6/9$. Subtask + AQ (ours) immediately resets insertion progress while retaining the $6/9$ from completed subtasks, then tracks recovery.

Annotators mark subtask boundaries and intervals of *suboptimal* or *regressed* execution; unmarked intervals are *nominal*. Nominal execution follows the task normally. Suboptimal execution is degraded without physical failure, such as a misaligned screw that remains held. Regressed execution loses local progress through physical failure, such as dropping the screw. Separately, each subtask receives a binary outcome: 1 for successful completion and 0 for failure. These outcomes describe how the subtask ends: a completed subtask can still contain regressed intervals followed by recovery.

The annotations define a local progress target $p_t \in [0, 1]$ without requiring numerical scores for individual frames. Within each uninterrupted attempt, we distribute progress according to action quality, giving nominal frames larger increments than suboptimal frames. In a successful subtask with retries, regression resets local progress to zero, and a subsequent attempt rebuilds it. The binary outcome anchors the final local-progress target.

Let $k_t^\star$ denote the annotated subtask to evaluate. We construct the whole-task progress target $P_t^\star = (k_t^\star + p_t)/K$ using the same composition as at inference. In the insertion retry in Fig. 3, the six completed subtasks retain their credit while insertion fails and recovers.

3.3. Training Subtask Selection and Progress Prediction

Because PRISM selects its own subtask, progress training must account for a description that is chosen incorrectly. For example, if insertion is only partly complete but the model selects fastening, fastening should receive zero progress rather than insertion's partial progress. We use task order to define the target for each queried subtask: earlier completed subtasks receive one, the annotated unfinished subtask receives its local progress, and later subtasks receive zero:

$$y_t(k_t) = \begin{cases} 1, & k_t < k_t^\star, \\ p_t, & k_t = k_t^\star, \\ 0, & k_t > k_t^\star. \end{cases} \tag{2}$$

Pretraining combines subtask selection with annotated-subtask and whole-task progress prediction.

Selection learns to identify $k_t^\star$, while local-progress training teaches the predictor to evaluate its description. Whole-task prediction provides auxiliary supervision from available overall labels, allowing data without reliable subtask annotations to contribute. During adaptation, we also query the progress predictor with descriptions selected by the model itself. For each selected description, Eq. 2 provides the corresponding local-progress target, including when the selected subtask is earlier or later than the annotated one.

We implement the shared predictor with Qwen3.5-4B [26] and LoRA [27], training the adapters, temporal transformer, and progress head while keeping the base VLM frozen. Selection and progress use cross-entropy objectives. For progress training, scalar targets are encoded as distributions over neighboring bins; the head predicts a distribution whose mean gives a continuous progress estimate. Both objectives place greater weight on predictions farther from the target. Pretraining and adaptation retain the same architecture and language interface. At inference, the model receives observations, the description, and the subtask list, without annotation inputs.

3.4. Using Progress for Policy Learning

Progress differences provide feedback about recorded actions: an increase indicates advancement, while a decrease indicates a setback. We use these differences in two ways, holding the conversion procedure fixed across reward models in each comparison.

Offline reinforcement learning

Using the learned progress model $\hat{P}(s)$, we assign each H-step action chunk $a_{t:t+H}$ a reward equal to the discounted sum of predicted progress increments:

$$r_t(s_t, a_{t:t+H}) = \sum_{j=0}^{H-1} \gamma^j \left[\hat{P}(s_{t+j+1}) - \hat{P}(s_{t+j})\right],$$

where γ is the per-step discount factor. Using these rewards, IQL [23] learns an action-value function $Q_\phi(s, a_{t:t+H})$ and a state-value function $V_\psi(s)$. We use the resulting advantage estimates, $A(s, a_{t:t+H}) = Q_\phi(s, a_{t:t+H}) - V_\psi(s)$, to update the policy with BPPO [24].

Progress-weighted behavior cloning

Even successful trajectories can contain unhelpful actions, so we use progress changes to weight their contribution to imitation learning. We map these progress changes, $\hat{P}(s_{t+1}) - \hat{P}(s_t)$, over each action step to high, intermediate, or low nonnegative imitation weights, using separate thresholds for human and policy actions. The weighted action loss is normalized by the total weight. Appendix A gives the complete target-construction rules and a worked failure-and-recovery example. Appendices B--D specify the model, training settings, and policy-learning conversions.

4. Experiments

Our experiments address four questions:

1. **Supervision:** Can structured subtask and quality annotations provide practical and useful progress supervision?
2. **Generalization:** How well does PRISM generalize across tasks, horizons, embodiments, and datasets?
3. **Pretraining and composition:** How does reward-model pretraining improve long-horizon adaptation, and does subtask composition improve progress prediction in both PRISM and a frontier VLM such as GPT-6 Astra [28]?
4. **Policy learning:** Do the resulting progress estimates improve downstream policy learning?

We first test the supervision used to train the models (Sec. 4.2), then evaluate zero-shot prediction, adaptation, and predictor reuse (Sec. 4.3), examine subtask composition (Sec. 4.4), and test policy learning (Sec. 4.5).

4.1. Experimental Setup

Zero-shot evaluation uses 50 held-out tasks, while long-horizon adaptation uses 3--5-minute airplane model assembly ($K = 9$) and car model assembly ($K = 10$) tasks.

We use RMSE to measure errors in predicted progress against constructed curves that preserve completed work and capture failures and recovery. Across ours and 3 other baseline reward models, lower aggregate RMSE on the 50 held-out tasks corresponds to higher or tied success on both downstream tasks (Sec. 4.5.1), supporting its use to prioritize models for policy evaluation. Long-task errors use subtask units (K times normalized RMSE on $[0, 1]$).

Policy experiments use our DreamZero World Action Model (WAM) reimplementation [29], holding initialization, data, and learning procedure fixed within each task and comparison while varying reward models.

4.2. Annotation Quality and Supervision Utility

We first measure annotation effort, then examine the resulting target curves, and finally test whether these targets improve reward-model learning.

Table 1: Annotation acceptance and time (minutes per video minute; mean $\pm$ SD).

Annotation	**Accept.↑**	**Time↓**
Direct points	0.76	2.6 ± 0.6
AQ only	0.90	1.4 ± 0.3
Subtask only	0.96	1.1 ± 0.2
Subtask + AQ (ours)	0.90	1.5 ± 0.4

Table 2: RoboMeter progress consistency under different supervision. D--U denotes Down--Up.

	Airplane		**Car**	
Supervision	Q-Rank↑	D--U↑	Q-Rank↑	D--U↑
Official [2]	69.0	57.9	44.5	68.5
Original target	63.9	59.7	76.0	49.3
Our target	**83.8**	**70.2**	**87.0**	**84.9**

4.2.1. Annotation Interfaces

To measure annotation effort and protocol acceptance, we compare numerical scoring (Direct points), subtask boundaries (Subtask only), action-quality windows (AQ only), and their combination (Subtask + AQ). Quality windows mark suboptimal or regressed execution; unmarked windows are nominal, as defined in Sec. 3.2. The study uses 100 trajectories and 12 annotators, each producing one annotation per trajectory, with each annotation jointly assessed by two additional reviewers. Acceptance is the fraction that follows the predefined protocol without revision. Subtask + AQ raises acceptance from 0.76 to 0.90 and reduces mean annotation time from 2.6 to 1.5 minutes per video minute (Table 1).

Subtask only is faster and has higher acceptance, but boundaries alone do not describe failures within an unfinished subtask. On clean successful trajectories without suboptimal or regressed segments, it can match Subtask + AQ given the same boundaries.

We next compare progress reconstruction from sparse annotations using the accumulation rule used in prior work [7], subtask or action-quality annotations alone, and their combination (Fig. 3). Together with Table 1, this comparison motivates Subtask + AQ: it captures sudden losses of local progress while preserving completed subtasks, and achieves higher efficiency and protocol acceptance than direct numerical scoring.

4.2.2. Progress Consistency under Different Supervision

We evaluate RoboMeter using human-labeled quality ordering (Q-Rank) and score decreases followed by increases around regressed events (Down--Up). Constructed-target finetuning achieves the highest scores on both tasks (Table 2).

4.3. Reward Prediction and Generalization

We evaluate zero-shot prediction on held-out tasks, adaptation to longer tasks and a different embodiment, and whole-trajectory ranking on a public benchmark.

4.3.1. Zero-Shot Prediction and Pretraining Coverage

On the 50 held-out tasks without target-task reward labels or fine-tuning, PRISM achieves RMSE 0.180 versus 0.227 for the strongest baseline, RoboMeter, a 20.7% reduction (Fig. 4(a)). Increasing pretraining from 15 to 200 tasks reduces held-out RMSE by 35.3% (Fig. 4(b)).

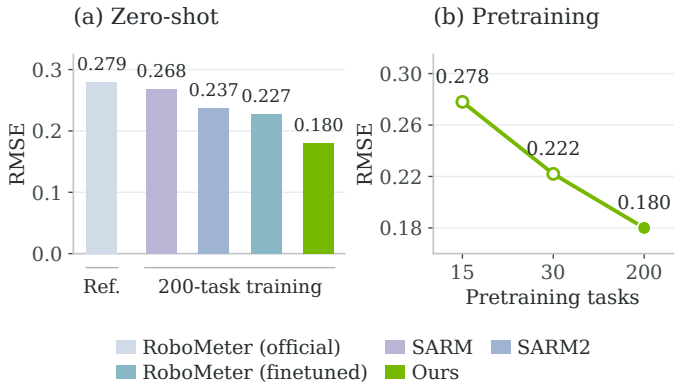


Figure 4: Zero-shot progress prediction and pretraining scale. Both panels report RMSE on the same 50 held-out tasks without target-task reward labels or fine-tuning. (a) Models trained on the shared 200-task corpus, with the released RoboMeter checkpoint (official) shown as an external reference. RoboMeter (finetuned) continues training from this checkpoint. (b) PRISM trained on nested sets of 15, 30, and 200 tasks, with training compute and other settings fixed. Task budgets are displayed as equally spaced conditions. Lower RMSE is better; the panels use different y-axis scales.

4.3.2. Long-Horizon Adaptation

We adapt each model to the plane-model and car-model assembly using 15 (D15) or 100 (D100) training trajectories per task, sharing annotations and constructed global progress targets. PRISM achieves the lowest RMSE in all four task--budget settings and, with D15, outperforms every baseline using D100 (Table 3). Averaged in subtask units across tasks, RMSE is 37.7% below SARM2, the strongest baseline, at both budgets. Section 4.3.4 analyzes pretrained initialization.

Table 3: Long-task adaptation on airplane and car model assembly. RMSE is in subtask units; D15/D100 use 15/100 labeled trajectories.

	Airplane ($K = 9$)		Car ($K = 10$)	
Method	**D15↓**	**D100↓**	**D15↓**	**D100↓**
RoboMeter	0.493	0.487	0.658	0.549
SARM	0.747	0.485	1.124	0.539
SARM2	0.498	0.461	0.629	0.472
Ours: no-pretrain	0.382	0.311	0.502	0.445
Ours	**0.308**	**0.262**	**0.394**	**0.319**

4.3.3. Cross-Embodiment Adaptation

We transfer reward models fine-tuned for airplane model assembly on the YAM robot to the same task on the Tianji robot, using 10 Tianji trajectories, and evaluate them on the same 30 Tianji test trajectories. PRISM reaches RMSE 0.452 versus 0.556 for the strongest baseline, RoboMeter, an 18.7% reduction (Table 4).

Table 4: YAM to Tianji adaptation: 10 labeled trajectories (D10); RMSE on the 0--9 scale.

	SARM	SARM2	RoboMeter	Ours
RMSE↓	0.919	1.167	0.556	**0.452**

4.3.4. Reusing Pretrained Knowledge through Language

We next examine how each model reuses knowledge learned from shorter tasks in these long-horizon settings. PRISM reuses its complete pretrained predictor through the same language interface,

whereas SARM replaces its task-specific classifier and SARM2 requires manual vocabulary mapping for full reuse. For PRISM, we compare pretrained initialization with base Qwen3.5-4B while keeping the architecture, target data, adaptation procedure, and evaluation fixed.

Table 5: Predictor reuse. Relative RMSE change (%) from each method's baseline without reward-model pretraining; negative is better.

		Airplane		Car		
Method	**Interface adaptation**	**D15↓**	**D100↓**	**D15↓**	**D100↓**	**Better**
Full predictor reuse						
Ours	**None required**	-19.4	-15.8	-21.5	-28.3	**4/4**
SARM	Unsupported	---	---	---	---	N/A
SARM2	Manual mapping	-9.7	$+4.4$	-18.2	$+2.2$	2/4
Partial predictor reuse						
SARM	Adapt classifier	$+24.4$	-28.7	-10.0	-1.8	3/4
SARM2	Adapt classifier	-2.9	$+22.2$	-9.1	$+15.6$	2/4

D15 and D100 denote labeled target-trajectory budgets. *Better* counts negative RMSE changes across the four task--budget settings. All methods receive subtask descriptions and target-task training. *None required* means no additional task-interface changes. SARM's task-specific subtask classifier prevents full predictor reuse across different subtask sets; N/A and dashes indicate this architectural limitation.

Pretraining reduces PRISM's RMSE by 15.8--28.3%, improving all four task--budget settings under full predictor reuse, compared with two or three for the tested baseline configurations (Table 5).

4.3.5. Trajectory Ranking on a Public Benchmark

We next test whether the learned progress model can rank complete trajectories on an external benchmark. Following the standard protocol in RBM-EVAL-OOD [2], we report Kendall scores to measure agreement with the trajectory-quality ordering (failure $<$ suboptimal $<$ successful) via progress-only supervision fine-tuning.

Without reward-model pretraining, switching the backbone from Qwen3-VL to Qwen3.5-4B improves the macro ranking score only slightly, from 0.519 to 0.529 (Table 6). Keeping Qwen3.5-4B fixed, pretraining with dense progress supervision on data from a different distribution brings a much larger gain, from 0.529 to 0.688, after the same progress-only fine-tuning on RBM-1M. Both RoboMeter rows, including per-dataset values, are quoted from its paper.

Table 6: RBM-EVAL-OOD ranking. Macro averages six domains; RoboMeter results quote arXiv v1 [2].

Method / initialization	**USC-F**	**USC-K**	**USC-T**	**USC-X**	**MIT-F**	**UTD-S**	**Macro↑**
mixed-objective supervision							
RoboMeter: official [2]	**0.646**	0.471	0.653	**0.694**	**0.601**	0.867	0.655
progress-only supervision							
RoboMeter: progress-only [2]	0.083	0.231	0.333	0.389	0.183	0.533	0.292
Ours: no-pretrain (Qwen3-VL)	0.333	0.456	0.389	0.500	0.504	**0.933**	0.519
Ours: no-pretrain (Qwen3.5-4B)	0.333	0.456	0.389	0.500	0.560	**0.933**	0.529
Ours	0.583	**0.560**	**0.917**	0.639	0.496	**0.933**	**0.688**

4.4. Subtask Composition and Progress Prediction

We test whether subtask composition improves progress prediction, then examine the roles of subtask selection and local-progress prediction.

4.4.1. Subtask Composition

On airplane model assembly, direct whole-task and composed predictors start from base Qwen without reward-model pretraining and use matched adaptation budgets. Automatic subtask selection and local-progress composition reduce RMSE by 17.3% with D15 and 13.4% with D100, including selection errors (Table 7).

Table 7: Subtask composition on airplane model assembly.

Configuration	D15↓	D100↓
Direct whole-task	0.462	0.359
Subtask composition	**0.382**	**0.311**

In zero-shot car model assembly, composition reduces accumulated RMSE from 1.578 to 0.524 for GPT-6 and by 32.4% for base Qwen3.5-4B (Table 8). These gains show that our proposed subtask composition improves progress prediction in both models. However, under OpenAI's standard API pricing, GPT-6 scoring in compositional inference mode costs approximately US$38.08 per trajectory in our setup, even at only 1 frame per second for trajectories averaging 340 seconds. This per-trajectory cost creates a substantial financial barrier to the repeated reward evaluations required for large-scale RL. In contrast, PRISM's 4B reward model runs locally on a single GPU with more than 10 GB of VRAM, avoiding recurring API charges and making repeated reward evaluation substantially more practical for RL applications.

4.4.2. Subtask Selection and Local Progress

Finally, we examine subtask selection and local-progress prediction in the zero-shot car model results. Using its own subtask predictions without car model adaptation, pretrained PRISM reaches 69.46% subtask selection accuracy and RMSE 1.3803, versus 25.37% and 3.0676 for base Qwen3.5-4B with composition. For the GPT-selected variant, pretrained PRISM uses the same subtask predictions as GPT-6 with composition. Holding GPT-6's subtask predictions fixed, replacing its local-progress estimates with those from pretrained PRISM further reduces accumulated RMSE from 0.524 to 0.494, without car model adaptation (Table 8). This result highlights the value of PRISM's learned within-subtask progress prediction under the same subtask selector.

Table 8: Long-horizon progress prediction on car model assembly. Base Qwen is Qwen3.5-4B; zero-shot Ours uses reward-model pretraining.

Model / procedure	Sel. acc.↑	RMSE↓
Zero-shot		
GPT-6: direct	---	1.578
GPT-6: composition (ours)	**86.56%**	0.524
Base Qwen: direct	---	4.541
Base Qwen: composition (ours)	25.37%	3.068
Ours: self-selected	69.46%	1.380
Ours: GPT-selected	**86.56%**	**0.494**
RoboMeter: GPT-selected	**86.56%**	0.624
Task-related adaptation (15 trajs)		
Ours: no-pretrain + adapt.	93.33%	0.502
Ours: pretrained + adapt.	**94.70%**	**0.394**

4.5. Downstream Policy Improvement

We test whether predicted progress improves policies through offline reinforcement learning and progress-weighted behavior cloning.

4.5.1. Zero-Shot Rewards for Offline Reinforcement Learning

Each reward source scores the same BC-policy rollouts on two held-out tasks: 50 for star insertion (one subtask) and 45 for tube transfer from a box to a rack (three subtasks). We convert predicted progress into rewards using the same rule across all reward models. These rewards supervise IQL critic learning [23], followed by BPPO actor updates [24]. Zero-shot means reward models receive no target-task reward annotations; policies train on scored target-task rollouts.

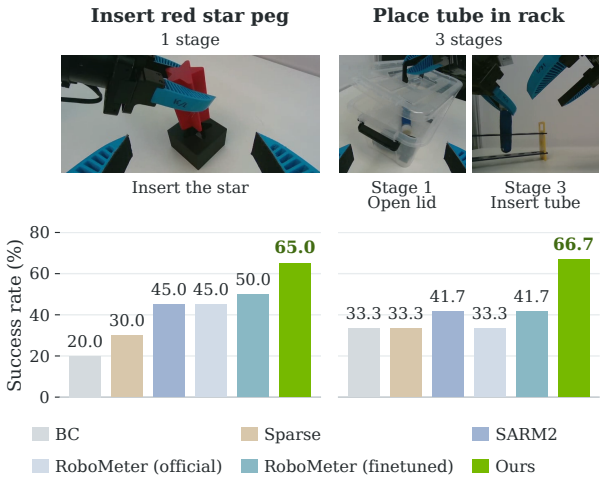


Figure 5: Policy success with zero-shot reward models on two held-out tasks. Each policy configuration is evaluated over 20 trials for star insertion and 12 for tube transfer. Across the four reward sources shared with Fig. 4(a), lower RMSE corresponds to higher or tied success on both tasks.

PRISM yields the highest observed success, 65.0% on star insertion and 66.7% on tube transfer, versus 50.0% and 41.7% for the strongest competing reward sources (Fig. 5). Across the four reward sources shared with Fig. 4(a), lower aggregate zero-shot RMSE on the 50 held-out tasks corresponds to higher or tied success on both downstream tasks. This observed association supports using held-out progress RMSE to prioritize promising reward models for downstream policy evaluation.

4.5.2. Progress-Weighted Learning on Long-Horizon Tasks

On the airplane and car model assembly, adapted reward models weight recorded actions using the progress-difference procedure in Sec. 3.4. We use a progress-delta reweighting scheme similar to SARM's reward-aligned BC [1], keeping the weighting procedure fixed across reward-model comparisons. We compare PRISM with unweighted learning and SARM2, the strongest baseline by average long-task RMSE.

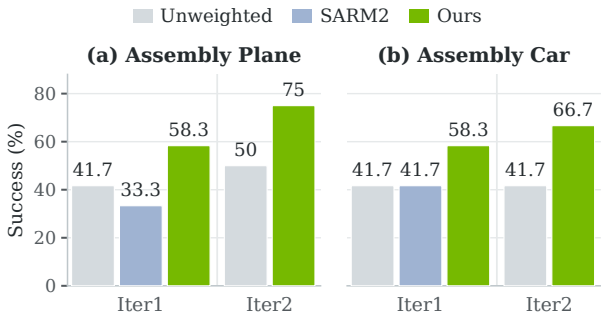


Figure 6: **Progress-weighted policy learning on long tasks.** Each reported success rate uses 12 evaluation trials. Comparisons share the task-specific parent policy within each iteration.

In Iter1, we test whether progress weighting improves learning from shared demonstrations and policy rollouts. For each task, a BC policy trained on 20 demonstrations (25% success) provides the common Iter1 initialization and collects 12 shared DAgger trajectories. All conditions train on the original demonstrations plus these trajectories, using these progress-based weights or unweighted BC. PRISM reaches 58% success on both tasks, versus 42% for unweighted BC (plain DAgger) and 33%/42% for SARM2 on airplane model and car model assembly, respectively.

In Iter2, we ask whether progress weighting still helps when all newly collected rollouts succeed. We select the checkpoint with the highest observed Iter1 success rate for each task (58%) as the shared parent for weighted and unweighted cloning. Both conditions train on the original 20 demonstrations plus 12 newly collected successful rollouts from this parent, excluding the Iter1 DAgger trajectories; PRISM scores these rollouts for the same delta-progress weighting procedure. Weighted cloning achieves 75% versus 50% on the airplane model task and 67% versus 42% on the

car model task (Fig. 6). Since all 12 new rollouts succeed, terminal outcomes cannot distinguish useful from unhelpful chunks within these trajectories; the results support dense progress changes for local credit assignment.

5. Conclusion

In this paper, we introduce PRISM, a generalizable reward model for robotics. It combines structured progress supervision with a shared predictor conditioned on the subtask that should be the focus of evaluation. Subtask boundaries and action-chunk judgments determine targets that capture failure resets and recovery through retries, and these targets also improve the existing RoboMeter architecture. Pretraining on short tasks supports zero-shot reward prediction, long-task adaptation, and public OOD ranking after adaptation. At the same time, the resulting progress estimates improve policies through offline RL and progress-weighted behavior cloning. The observed agreement between rankings by held-out progress RMSE and policy success supports using RMSE to prioritize promising reward models for downstream evaluation. The long-horizon comparisons show benefits from subtask-based inference in both GPT-6 and base Qwen3.5-4B, and from pretrained initialization of the 4B predictor under matched target-task adaptation.

Acknowledgments

We used AI, including GPT-6 Astra, Claude Fable 5, and Opus 5, to polish the language, improve the presentation clarity of all figures, and assist with code implementation.

References

[1] Qianzhong Chen, Justin Yu, Mac Schwager, Pieter Abbeel, Fred Shentu, and Philipp Wu. SARM: Stage-aware reward modeling for long horizon robot manipulation, 2025. 2, 3, 11

[2] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, Minyoung Hwang, Abrar Anwar, Sidhant Kaushik, Aditya Shah, Alex S. Huang, Luke Zettlemoyer, Dieter Fox, Yu Xiang, Anqi Li, Andreea Bobu, Abhishek Gupta, Stephen Tu, Erdem Biyik, and Jesse Zhang. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons, 2026. 2, 3, 7, 9

[3] Minttu Alakuijala, Reginald McLean, Isaac Woungang, Nariman Farsad, Samuel Kaski, Pekka Marttinen, and Kai Yuan. Video-Language Critic: Transferable Reward Functions for Language-Conditioned Robotics. *Transactions on Machine Learning Research*, 2025. 3

[4] Tony Lee, Andrew Wagenmaker, Karl Pertsch, Percy Liang, Sergey Levine, and Chelsea Finn. RoboReward: General-purpose vision-language reward models for robotics, 2026. 2, 3

[5] Shirui Chen, Cole Harrison, Ying-Chun Lee, Angela Jin Yang, Zhongzheng Ren, Lillian J. Ratliff, Jiafei Duan, Dieter Fox, and Ranjay Krishna. TOPReward: Token probabilities as hidden zero-shot rewards for robotics, 2026. 2, 3

[6] Qianzhong Chen, Hau Zheng, Justin Yu, Suning Huang, Jiankai Sun, Ken Goldberg, Chuan Wen, Pieter Abbeel, Yide Shentu, Philipp Wu, and Mac Schwager. SARM2: Multi-task stage aware reward modeling for self improving robotic manipulation, 2026. 2, 3

[7] Yiming Mao, Zixi Yu, Weixin Mao, Yinhao Li, Qirui Hu, Zihan Lan, Minzhao Zhu, and Hua Chen. ARM: Advantage reward modeling for long-horizon manipulation, 2026. 2, 3, 5, 7

[8] Yecheng Jason Ma, Shagun Sodhani, Dinesh Jayaraman, Osbert Bastani, Vikash Kumar, and Amy Zhang. VIP: Towards Universal Visual Reward and Representation via Value-Implicit Pre-Training. In *International Conference on Learning Representations*, 2023. 3

[9] Yecheng Jason Ma, William Liang, Vaidehi Som, Vikash Kumar, Amy Zhang, Osbert Bastani, and Dinesh Jayaraman. Liv: Language-image representations and rewards for robotic control, 2023.

[10] Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-Language Models are Zero-Shot Reward Models for Reinforcement Learning. In *International Conference on Learning Representations*, 2024.

[11] Sumedh Sontakke, Jesse Zhang, Séb Arnold, Karl Pertsch, Erdem Bıyık, Dorsa Sadigh, Chelsea Finn, and Laurent Itti. RoboCLIP: One Demonstration is Enough to Learn Robot Policies. In *Advances in Neural Information Processing Systems*, volume 36, 2023. 3

[12] Yecheng Jason Ma, Joey Hejna, Ayzaan Wahid, Chuyuan Fu, Dhruv Shah, Jacky Liang, Zhuo Xu, Sean Kirmani, Peng Xu, Danny Driess, Ted Xiao, Jonathan Tompson, Osbert Bastani, Dinesh Jayaraman, Wenhao Yu, Tingnan Zhang, Dorsa Sadigh, and Fei Xia. Vision language models are in-context value learners, 2024. 3

[13] Dongchi Huang, Hongyin Zhang, Bohan Hou, Siteng Huang, Zhian Su, Hang Guo, Tong Lu, Zhaofeng Xu, Jiahao Tang, Jianfei Yang, Donglin Wang, Peixi Peng, Mingxiu Chen, Deli Zhao, and Xin Li. RynnValue: Scaling robotic value foundation models with temporal distance, 2026. 3

[14] Jiahui Zhang, Yusen Luo, Abrar Anwar, Sumedh Anand Sontakke, Joseph J Lim, Jesse Thomason, Erdem Biyik, and Jesse Zhang. ReWiND: Language-guided rewards teach robot policies without new demonstrations, 2025. 3

[15] Justin Yu, Andrew Goldberg, Kavish Kondap, Karim El-Refai, Ethan Ransing, Qianzhong Chen, Mac Schwager, Fred Shentu, Philipp Wu, and Ken Goldberg. WARP-RM: A warp-augmented relative progress reward model for data curation, 2026. 3

[16] Yuyang Liu, Chuan Wen, Yihang Hu, Dinesh Jayaraman, and Yang Gao. TimeRewarder: Learning dense reward from passive videos via frame-wise temporal distance, 2026. ICML 2026 spotlight; first preprint 2025. 3

[17] Haoyu Zhang, Zecui Zeng, Bin Wang, Lusong Li, Liang Lin, and Long Cheng. Dream2Reward: Transition-Alignment Reward Models from Positive Demonstrations for Robotic Manipulation, 2026. 3

[18] Huajie Tan, Sixiang Chen, Yijie Xu, Zixiao Wang, Yuheng Ji, Cheng Chi, Yaoxu Lyu, Zhongxia Zhao, Xiansheng Chen, Peterson Co, Shaoxuan Xie, Guocai Yao, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Robo-Dopamine: General process reward modeling for high-precision robotic manipulation, 2025. 3

[19] Yu Fang, Wanxi Dong, Jiaqi Liu, Yue Yang, Mingxiao Huo, Yao Mu, Huaxiu Yao, Li Erran Li, Daniel Szafir, and Mingyu Ding. DenseReward: Dense reward learning via failure synthesis for robotic manipulation, 2026. 3

[20] Changyeon Kim, Minho Heo, Doohyun Lee, Honglak Lee, Jinwoo Shin, Joseph J. Lim, and Kimin Lee. Subtask-aware visual reward learning from segmented demonstrations. In *International Conference on Learning Representations*, 2025. 3

[21] Xiuhui Zhang, Ning Gao, Xingyu Jiang, Yihui Chen, Yuheng Pan, Mohan Zhang, and Yue Deng. Progress reward model for reinforcement learning via large language models. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 100745--100772, 2025. doi: 10.52202/085713-3370. 3

[22] Shaopeng Zhai, Qi Zhang, Tianyi Zhang, Fuxian Huang, Haoran Zhang, Ming Zhou, Shengzhe Zhang, Litao Liu, Sixu Lin, and Jiangmiao Pang. A Vision-Language-Action-Critic Model for Robotic Real-World Reinforcement Learning, 2025. 3

[23] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline Reinforcement Learning with Implicit Q-Learning. In *International Conference on Learning Representations*, 2022. 3, 6, 11

[24] Zifeng Zhuang, Kun Lei, Jinxin Liu, Donglin Wang, and Yilang Guo. Behavior Proximal Policy Optimization. *arXiv preprint arXiv:2302.11312*, 2023. 6, 11

[25] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning, 2019. 3

[26] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. 6

[27] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. 6

[28] OpenAI. GPT-6 Astra. OpenAI API documentation, 2026. Accessed: 2026-09-16. 6

[29] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World Action Models are Zero-shot Policies, 2026. 7

A. Progress-Target Construction

This appendix expands the structured supervision in Sec. 3.2. We give the annotation inputs, the complete local-target construction, and the screw-insertion example underlying Fig. 3.

A.1. Annotation Inputs and Progress Composition

Annotators provide subtask boundaries, action-quality (AQ) intervals, and a binary outcome for each subtask. They mark intervals of suboptimal or regressed execution; unmarked intervals are nominal. Nominal execution follows the task normally. Suboptimal execution is degraded without physical failure, such as a misaligned screw that remains held. Regressed execution loses local progress through physical failure, such as dropping the screw. The outcome $o_k \in \{0, 1\}$ records whether subtask k ends in failure or successful completion. A successfully completed subtask can contain regressed intervals followed by recovery. These annotations do not require numerical progress scores for individual frames.

At global trajectory time t, let $k_t^\star$ be the zero-based annotated subtask index, $p_t \in [0, 1]$ its local progress target, and K the number of subtasks. The whole-task target is

$$P_t^\star = \frac{k_t^\star + p_t}{K}. \tag{3}$$

Each completed subtask contributes $1/K$. At a completed boundary, the states $(k, 1)$ and $(k+1, 0)$ give the same whole-task progress. A local reset therefore preserves the credit for preceding completed subtasks, under the assumption in Sec. 3.1 that these subtasks remain completed.

A.2. Chunk-Wise Local Progress

The construction uses three parameters (Table 9). The ratio ρ controls how quickly nominal execution advances relative to suboptimal execution within an attempt. The cap p_{cap} and reference duration $T_{\min}$ determine how far an attempt can advance before an observed regression resets it.

Table 9: Parameters for local-progress construction. The cap and reference duration shown are for the screw-insertion subtask in Appendix A.3.

Symbol	Meaning	Value
$\rho > 1$	Nominal-to-suboptimal slope ratio	20
$0 < p_{\text{cap}} < 1$	Cap on the endpoint before regression	0.95 (screw insertion)
$T_{\min} > 0$	Fastest measured time to reach the cap	$9\,\text{s}$ (screw insertion)

Step 1: Locate and weight the chunk.
Subtask boundaries and regressed AQ reset local progress. Changes between nominal and suboptimal AQ do not reset it. For an uninterrupted attempt, let τ_{start} be its latest reset time and τ_{next} the next regression onset or subtask boundary. Both timestamps are measured in seconds from the start of the subtask. The full chunk duration and time elapsed from its reset are

$$T_{\text{chunk}} = \tau_{\text{next}} - \tau_{\text{start}}, \qquad t_{\text{chunk}} = t_{\text{subtask}} - \tau_{\text{start}}. \tag{4}$$

For each non-regressed update i in the chunk $\mathcal{C}$, assign

$$w_i = \begin{cases} \rho, & \text{nominal AQ}, \\ 1, & \text{suboptimal AQ}, \end{cases} \qquad N_{\text{chunk}} = \sum_{i \in \mathcal{C}} w_i \, \Delta t_{\text{chunk},i}, \tag{5}$$

where $\Delta t_{\text{chunk},i} = t_{\text{chunk},i} - t_{\text{chunk},i-1}$ and $t_{\text{chunk},0} = 0$ at the reset. Thus, N_{chunk} is the weighted duration of the non-regressed updates.

Step 2: Set the endpoint and slope.
Let p_{last} denote the target at the chunk's last non-regressed update. The endpoint depends on how the chunk ends:

$$p_{\text{last}} = \begin{cases} p_{\text{cap}} \min\left(1, \dfrac{T_{\text{chunk}}}{T_{\min}}\right), & \text{Case A: interrupted by regression,} \\ o_k, & \text{Case B: reaches the subtask boundary.} \end{cases} \tag{6}$$

In Case A, a chunk shorter than $T_{\min}$ reaches a proportionally smaller endpoint; a longer chunk reaches the cap. In Case B, the annotated subtask outcome anchors the endpoint. For either case, normalize the weighted updates using

$$s_{\text{chunk}} = \frac{p_{\text{last}}}{N_{\text{chunk}}}. \tag{7}$$

Step 3: Accumulate or reset.
During a regressed interval, hold local progress at zero. Otherwise, accumulate from the current chunk's reset:

$$p_{t_{\text{chunk},j}} = s_{\text{chunk}} \sum_{i=1}^{j} w_i \, \Delta t_{\text{chunk},i}. \tag{8}$$

Use this value as p_t at the corresponding global time in Eq. 3. Nominal and suboptimal updates share the chunk's endpoint normalization, but their slopes differ by the factor ρ. Recovery begins a new accumulation from zero while completed-subtask credit remains unchanged.

A.3. Parameter Selection and a Worked Example

For screw insertion, we compare successful demonstrations with deliberately induced failures across more than 20 diverse trials collected in advance. Failures occur both while picking up the screw and just before insertion into the hole. Comparing these events with successful demonstrations places the latest observed failures that reset local progress just below 0.95 progress, motivating $p_{\text{cap}} = 0.95$ for this subtask. The fastest observed time to reach this level is approximately $9\,\text{s}$, giving $T_{\min} = 9\,\text{s}$.

Figure 7 follows insertion, subtask 7 of 9 in airplane assembly. Six subtasks are already complete, so the whole-task target is $(6 + p_t)/9$. The example contains 1,117 synchronized samples at 30 frames per second, indexed from 0 through 1,116. The first attempt is nominal through frame 249; frames 250--820 are regressed; frames 821--1,116 show nominal recovery to a successful outcome.

Before failure: Case A.
Regression begins at $250/30 \approx 8.33\,\text{s}$, so the interrupted chunk has $T_{\text{chunk}} = 250/30\,\text{s}$. Equation 6 gives

$$p_{\text{last}} = 0.95 \frac{250/30}{9} \approx 0.8796. \tag{9}$$

The last non-regressed sample is frame 249, at $8.30\,\text{s}$. Normalizing over the non-regressed updates makes this sample reach p_{last} before the reset at frame 250. Because every update in this attempt is nominal, the common factor ρ cancels in Eq. 8.

Regression and recovery: Case B.
Frames 250--820 have $p_t = 0$, retaining whole-task progress $6/9$. Recovery accumulates from the last zero-valued sample at $820/30\,\text{s}$; its first positive update is frame 821. The 296 nominal recovery updates reach the subtask boundary at frame 1,116, or $37.20\,\text{s}$. The successful outcome sets $p_{\text{last}} = 1$, and whole-task progress reaches $7/9$.

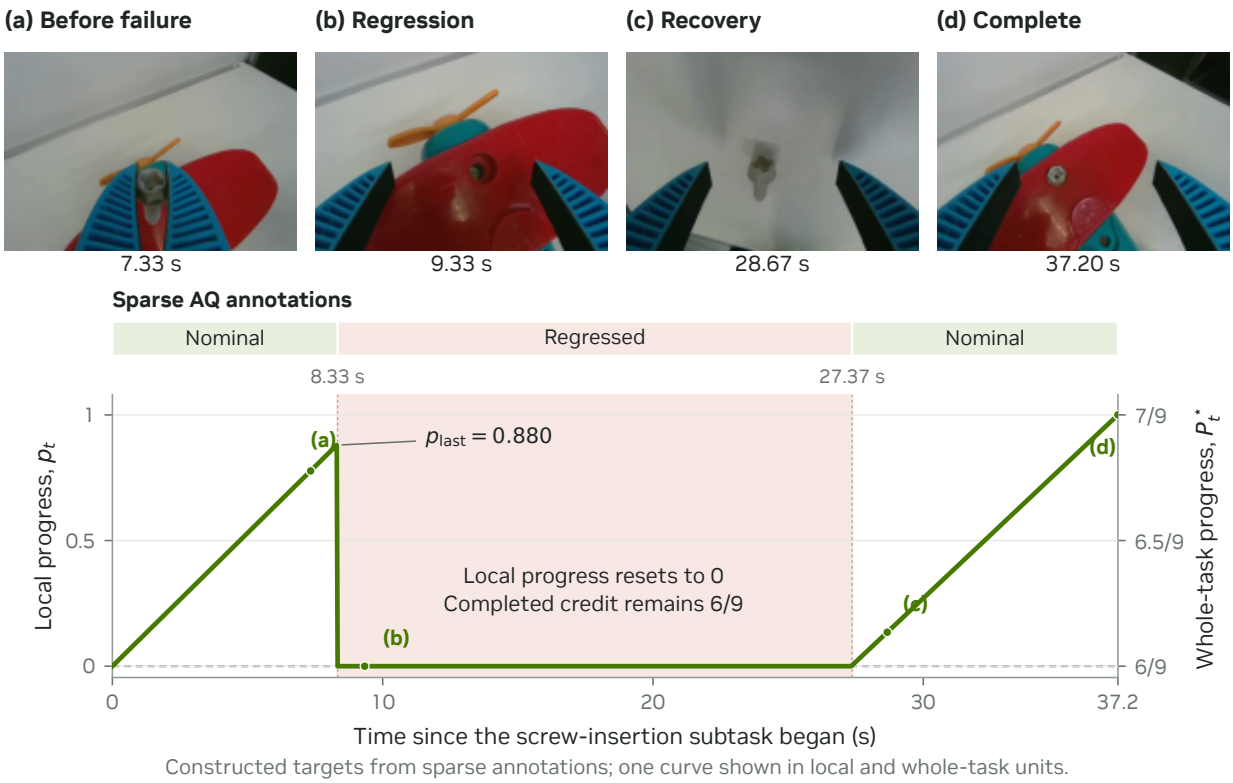


Figure 7: **Worked construction of a failure-and-recovery target.** Selected views from the synchronized screw-insertion recording accompany the AQ timeline and constructed progress target. The first attempt ends in regression (Case A), resetting local progress to zero and whole-task progress to $6/9$. Recovery reaches the successful subtask boundary (Case B), where local progress is one and whole-task progress is $7/9$. The curve shows annotation-derived supervision, not reward-model predictions; time is measured from the start of this subtask.

B. Reward Model and Training Details

This appendix gives the implementation settings for the shared predictor in Secs. 3.1 and 3.3. Table 10 summarizes the architecture, inputs, and optimization settings.

B.1. Architecture and Inputs

Subtask selection and local-progress prediction share the VLM, with a temporal transformer aggregating features for the progress head. The sampled visual context can include later observations; the policy-learning experiments use offline scoring of recorded trajectories.

B.2. Training and Inference

Pretraining and adaptation use the objectives in Sec. 3.3, retaining the same architecture and language interface. During adaptation, descriptions selected by the model itself receive the order-conditioned targets in Eq. 2.

Inference uses two sequential calls: the first selects a subtask from the supplied ordered list, and the second predicts local progress conditioned on that subtask's description. The selected index and local prediction are composed using Eq. 1. Inference requires observations and text inputs, without subtask-boundary, AQ, or outcome annotations.

Table 10: Reward-model implementation settings. The learning rate and batch size are shared across the compared reward-model training methods.

Setting	Value
Backbone	Qwen3.5-4B vision--language model
Frozen components	Base VLM weights and visual encoder
Trained components	Language-decoder LoRA adapters, temporal transformer, and progress head
LoRA rank	32
Text inputs	Task instruction and ordered subtask descriptions
Visual inputs	Eight sampled timesteps of synchronized camera views around the query time
Calls per scoring query	Two: subtask selection, then progress prediction for the selected description
Learning rate	2×10^{-5}
Batch size	128

C. Offline Reinforcement Learning Details

This appendix specifies the reward conversion and data used in the offline-RL experiments of Sec. 4.5.1. All reward sources score the same BC-policy rollouts within each task and use the same reward conversion. The reward models receive no target-task reward annotations; policy learning uses the scored target-task rollouts.

Reward conversion and policy update.
For each action chunk of horizon H, we sum discounted predicted whole-task progress increments:

$$r_t(s_t, a_{t:t+H}) = \sum_{j=0}^{H-1} \gamma^j \left[\hat{P}(s_{t+j+1}) - \hat{P}(s_{t+j})\right], \qquad H = 20, \quad \gamma = 0.997. \tag{10}$$

Here, γ is the per-step discount factor. These rewards supervise IQL action-value and state-value learning. The resulting advantages $A(s,a) = Q_\phi(s,a) - V_\psi(s)$ guide BPPO actor updates, following Sec. 3.4. Policy experiments use the DreamZero WAM reimplementation described in Sec. 4.1, with initialization, data, and learning procedure held fixed within each reward-model comparison.

Table 11: Offline-RL data and evaluation. Rollouts are shared across reward sources; evaluation counts are per policy configuration.

Task	Subtasks	Training rollouts	Evaluation trials
Star insertion	1	50	20
Tube transfer from box to rack	3	45	12

D. Progress-Weighted Behavior Cloning Details

This appendix specifies the long-horizon policy-learning procedure in Sec. 4.5.2. We use the D100 versions of PRISM and SARM2, each fine-tuned with 100 labeled target-task trajectories. Within each task and iteration, reward-model comparisons share the training data, parent policy, progress thresholds, and weighting rule.

D.1. Training Data and Parent Policies

Table 12 summarizes the protocol for airplane model assembly ($K = 9$) and red car model assembly ($K = 10$). In Iter1, all conditions start from the same BC parent and train on the demonstrations plus its shared DAgger trajectories. In Iter2, the checkpoint with the highest observed Iter1 success for each task is the common parent for weighted and unweighted cloning. The new rollouts all succeed and replace the Iter1 DAgger trajectories; the original demonstrations are retained. Each reported policy success rate uses 12 evaluation trials per task.

Table 12: Progress-weighted BC data and initialization, per task. Weighted and unweighted conditions share the parent and data within each iteration.

Iteration	Parent policy	Training data	Eval. trials
Iter1	BC on 20 demos	20 demonstrations + 12 shared DAgger trajectories	12
Iter2	Best Iter1 checkpoint	Original 20 demonstrations + 12 new successful rollouts	12

D.2. Progress Differences and Imitation Weights

Let $\widehat{P}_t \in [0, 1]$ be normalized whole-task progress at action time step t. We compute

$$\Delta\widehat{P}_t = \widehat{P}_{t+1} - \widehat{P}_t \tag{11}$$

between adjacent frames at 30 frames per second, an interval of $1/30\,\mathrm{s}$. Each action time step receives one imitation weight. High, intermediate, and low progress increments receive weights 1, 0.5, and 0, respectively, using the upper and lower thresholds in Table 13. Actions assigned zero weight contribute no imitation loss. For per-action imitation loss $\ell_t(\theta)$ and weights ω_t, the weighted objective is normalized by the total weight:

$$\mathcal{L}_{\mathrm{WBC}}(\theta) = \frac{\sum_t \omega_t\, \ell_t(\theta)}{\sum_t \omega_t}, \qquad \omega_t \in \{0, 0.5, 1\}. \tag{12}$$

Table 13: Progress-weighted BC thresholds and weights. Threshold pairs are ordered as [upper, lower], in normalized whole-task progress units per action step.

Setting	Value
Human-action thresholds	$[0.35,\ -0.13] \times 0.01/K$
Policy-action thresholds	$[0.5,\ -0.2] \times 0.01/K$
High / intermediate / low weights	$1.0\ /\ 0.5\ /\ 0$
Progress-difference interval	Adjacent frames at 30 fps ($1/30\,\mathrm{s}$)
Subtask count K	9 (airplane); 10 (red car)

Threshold selection.
Thresholds are set from average nominal increments in progress curves constructed from human annotations in the reward-model evaluation data. We then apply these thresholds to model-predicted increments on unseen policy-learning trajectories. Separate human-action and policy-action thresholds reflect the greater diversity of initial conditions and motion strategies in human demonstrations. The thresholds and weighting procedure remain fixed across the compared reward models.